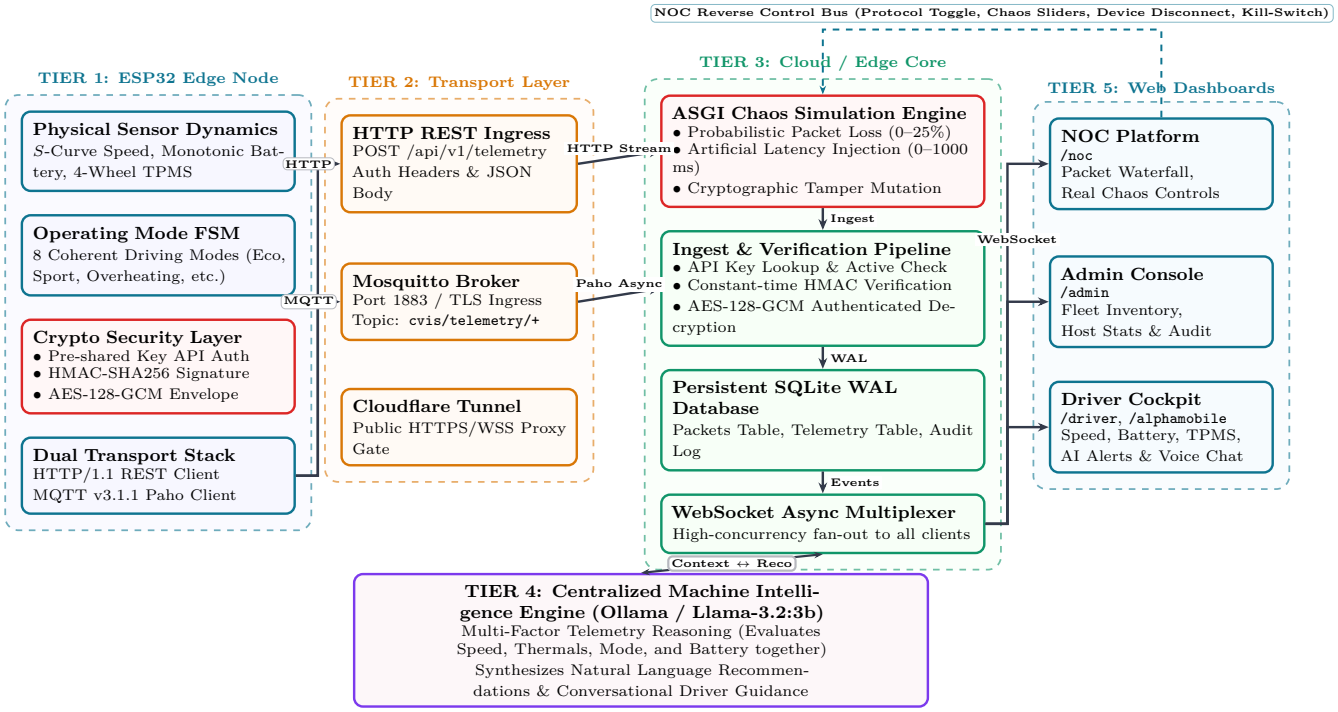
CONNECTED VEHICLE INTELLIGENCE SYSTEM (CVIS)

PAGE 1: SYSTEM ARCHITECTURE

Course: 21ECC402P — Computer Communication and Network Security | Faculty Technical Reference Guide

Justin Jacob Saju

Core Academic Thesis: Vehicles must **not** run heavy neural models directly on resource/thermally-constrained in-vehicle ECUs. The vehicle is designed as a **thin, hardened, cryptographically secure networked client**. The compute-intensive AI lives centrally in the cloud/edge data center. This project implements and demonstrates the **secure, fault-tolerant communication and network security layer** that makes this architecture viable, verifiable, and economically scalable.



Architectural Tier	Component Implementation	Primary Engineering Responsibility	Demonstrated CCNS Syllabus Concept
Tier 1: Vehicle Node	ESP32 / C++ Firmware / FreeRTOS	Sensor dynamics simulation, 8-mode FSM, HMAC	Lightweight client security, message integrity
Tier 2: Transport Layer	HTTP REST + Eclipse Mosquitto	Dual-protocol transmission, runtime agility	OSI Model, Protocol Interoperability, Transport trade-offs
Tier 3: Cloud / Backend	FastAPI, SQLite WAL, ASGI	Middleware chaos injection, crypto verification	Access Control, HMAC validation, DOS/Loss resilience
Tier 4: Intelligence	Ollama Local SLM (Llama-3.2:3b)	Multi-sensor semantic telemetry analysis	Cloud AI application layer riding securely on network
Tier 5: Presentation	Unified Next.js 16 (React, Tailwind)	Role-based Driver, NOC, and Admin monitoring	Network Operations Center (NOC), auditability, HUD

PROJECT WORKING & RUNTIME DATA FLOW LIFECYCLE

PAGE 2: RUNTIME FLOW

Course: 21ECC402P — Computer Communication and Network Security | Ingestion Pipeline

8-Stage Data Flow Lifecycle

Operational Lifecycle Overview: Telemetry follows a strict, verifiable 8-stage pipeline. The vehicle samples dynamic physics parameters, constructs an authenticated cryptographic frame, and dispatches it over the active network protocol. The cloud middleware applies software-defined network chaos (testing real-world degradation), validates identity and cryptographic integrity, persists raw payloads into write-ahead logs, invokes multi-factor artificial intelligence, and fans out updates to live operations dashboards in sub-5ms latency.

STAGE 1: Vehicle Telemetry Generation & Physical State Synthesis

ESP32 firmware calculates continuous velocity ($a = F/m$), thermal dissipation, monotonic battery drain, and 4-wheel TPMS.

1 → 2: Physical State Vector

STAGE 2: Canonical Wire Framing, Digital HMAC Signing & AES-GCM Encapsulation

Serializes JSON payload → computes HMAC-SHA256(K_{dev}, Payload) → packages envelope with 96-bit IV and 128-bit Auth Tag.

2 → 3: Signed Cryptographic Frame

STAGE 3: Dual-Stack Ingress Transmission (Active Protocol Dispatch)

Transmits frame via currently active NOC-selected protocol: HTTP POST with `X-API-Key` / `X-HMAC` **OR** MQTT on `cvis/telemetry/{id}`.

3 → 4: Ingress Network Stream

STAGE 4: ASGI Chaos Simulation Middleware (Fault Injection)

Intercepts raw ASGI stream before FastAPI handling: probabilistically drops packet (P_{loss}), injects latency (τ_{delay}), or mutates bits.

Simulated Drop
Loss trigger (P_{loss})
Node times out → Retries

4 → 5: Chaos-Passed Wire Frame

STAGE 5: Cryptographic Validation & Tamper Rejection

Checks database device active status → verifies HMAC signature using constant-time comparison. **Rejects with HTTP 401 if tampered.**

HTTP 401 Rejection
Signature mismatch
Tamper event logged in DB

5 → 6: Authenticated Telemetry

STAGE 6: Atomic SQLite Persistence (Write-Ahead Logging)

Records full raw packet metadata in `packets` table and parsed physical fields in `telemetry` table. Guarantees zero dropped records.

6 → 7: Multi-Sensor Context Vector

STAGE 7: Multi-Factor Machine Intelligence Analysis (Ollama Core)

Synthesizes complete multi-sensor vector: speed + motor thermals + battery state + driving mode. Evaluates composite anomaly risk.

7 → 8: AI Risk Diagnosis & Action

STAGE 8: Real-Time Asynchronous WebSocket Fan-Out & Tri-View Rendering

Broadcasts update via WebSocket manager to Driver Dashboard (`/driver`), NOC Operations Panel (`/noc`), and Admin Console (`/admin`).

Operational Characteristics: The pipeline guarantees end-to-end cryptographic integrity without blocking transport. SQLite WAL mode decouples disk commits from network I/O, allowing asynchronous WebSocket broadcast in sub-5ms latency. If the centralized AI engine is halted, Stages 1 through 6 and Stage 8 continue functioning seamlessly.

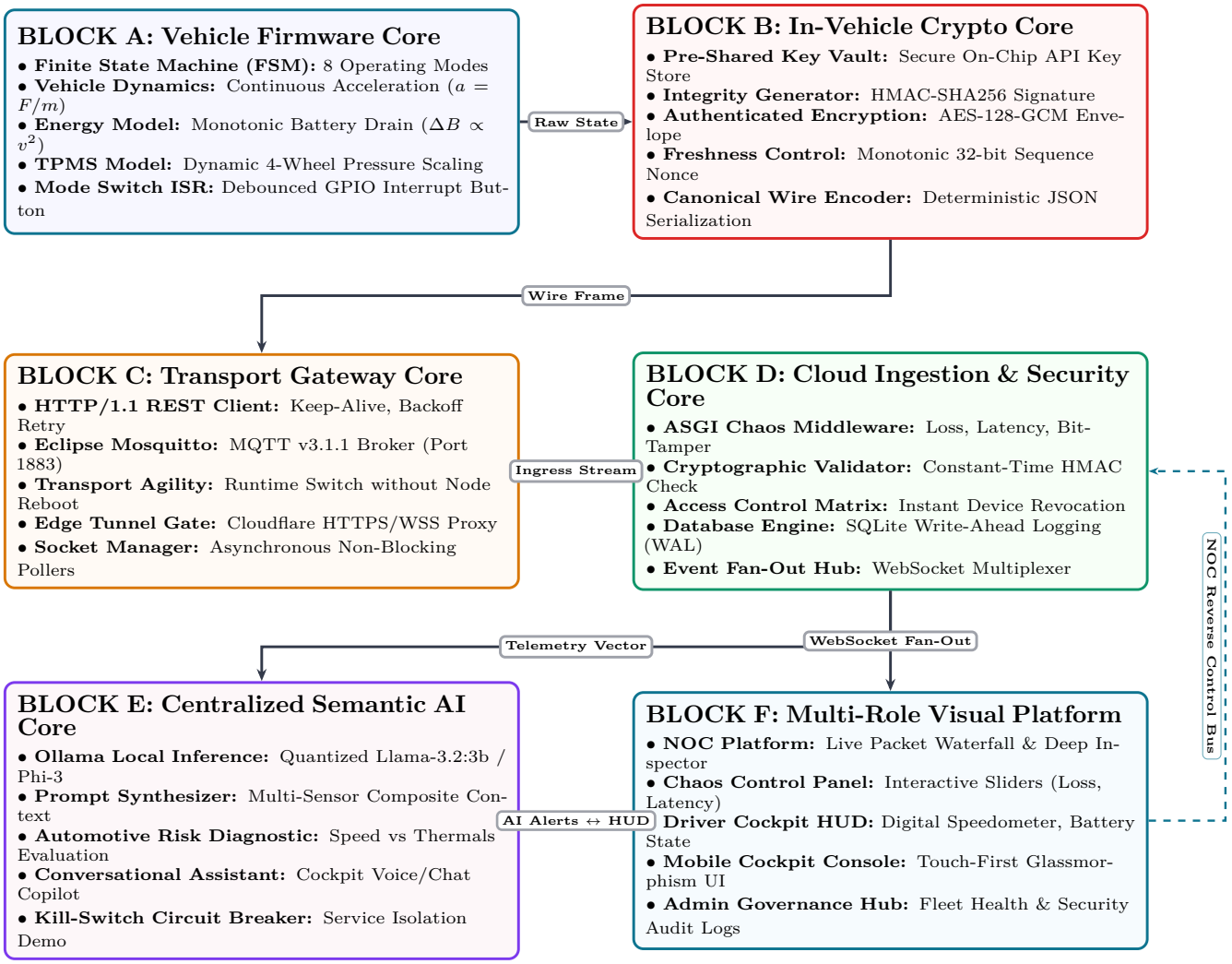
MODULAR FUNCTIONAL BLOCK DIAGRAM

PAGE 3: BLOCK DIAGRAM

Course: 21ECC402P — Computer Communication and Network Security | Subsystem Engineering

Modular Interface Matrix

Modular Subsystem Architecture: The CVIS architecture is decomposed into six cleanly decoupled functional blocks. Control signals, cryptographic keys, and telemetry streams pass across strictly typed, bounded interfaces. This block design ensures that failure or degradation in one module (such as transient AI downtime or cellular packet drops) does not compromise overall vehicular telemetry stability.



Subsystem Coupling Summary: Hardware boundaries separate Block A and B (Edge) from Block C, D, and E (Cloud Infrastructure). Interface contracts are enforced via strict JSON schema validation and cryptographic message authentication codes (MACs). Failure in presentation layers (Block F) or machine learning models (Block E) has zero impact on core telemetry capture or packet archival in Block D.

EXPLANATION OF MODULAR FUNCTIONAL BLOCKS

PAGE 4: BLOCK EXPLANATION

Course: 21ECC402P — Computer Communication and Network Security | Subsystem Analysis

Technical Subsystem Breakdown

Block A: Vehicle Firmware Core

Architectural Role: Represents the physical automotive edge client. Implemented in C++/FreeRTOS on ESP32 microcontrollers. It maintains an 8-mode Finite State Machine (FSM) cycling through realistic vehicle conditions: Healthy, Eco, Sport, Heavy Traffic, Low Battery, Battery Overheating, Charging, and Motor Fault.
Physics Model: Replaces stepped random snapshots with continuous kinematics:

$$a = \frac{F_{\text{traction}} - F_{\text{drag}}}{m}, \quad v_{t+\Delta t} = v_t + a\Delta t$$

Battery drainage is strictly monotonic ($\Delta B \propto v^2$), and dynamic 4-wheel TPMS pressures heat up with velocity (33.8 to 34.7 PSI).

Block B: In-Vehicle Crypto Core

Architectural Role: Provides client-side security before payloads touch untrusted cellular or WiFi links. It houses the device's unique pre-shared secret in a secure key store.
Security Mechanics: Computes an HMAC-SHA256 digital signature over the canonical JSON payload:

$$\text{HMAC}(K, M) = \text{SHA256}((K \oplus \text{opad}) \parallel \text{SHA256}((K \oplus \text{ipad}) \parallel M))$$

When encrypted mode is active, the payload is wrapped in an AES-128-GCM envelope carrying a 96-bit unique IV and 128-bit authentication tag.

Block C: Transport Gateway Subsystem

Architectural Role: Provides dual-stack transport layer agility between HTTP/1.1 REST and MQTT v3.1.1.
Network Tradeoffs: HTTP provides standard request-response semantics with HTTP keep-alive, ideal for passing through enterprise proxies (port 443). MQTT provides ultra-lightweight pub/sub streaming over Eclipse Mosquitto (port 1883), reducing per-packet header overhead by 92% (2-byte fixed header vs $\approx$ 450 bytes of ASCII HTTP headers). The system can dynamically toggle protocols at runtime without restarting vehicle hardware.

Block D: Cloud Ingestion & Security Core

Architectural Role: The central gateway hosted on the cloud/edge data center server (FastAPI ASGI core).
Ingestion Pipeline: 1. Intercepts incoming requests in `ChaosMiddleware` to inject probabilistic packet drops, latency, or bit corruption. 2. Performs constant-time cryptographic verification (`hmac.compare_digest()`) to prevent timing side-channel attacks. 3. Persists raw packets into SQLite in Write-Ahead Logging (WAL) mode to guarantee zero concurrency locks. 4. Fans out validated events to connected WebSocket clients.

Block E: Centralized Semantic AI Subsystem

Architectural Role: Houses the compute-intensive artificial intelligence layer, completely decoupled from in-vehicle hardware.
Inference Pipeline: Leverages a local quantized Small Language Model (SLM) via Ollama (`llama3.2:3b` or `phi3-mini`). Rather than simplistic single-parameter threshold rules, the engine constructs a multi-sensor prompt context vector (Speed, Motor Temp, Battery %, Mode, and TPMS) and evaluates composite risks (e.g., detecting motor stress during sport mode vs cooling failures).

Block F: Multi-Role Visualization Platform

Architectural Role: Unified Next.js 16 frontend delivering three role-gated operational surfaces over a single multiplexed WebSocket feed:

- **NOC Platform (`/noc`):** Real-time packet waterfall, click-to-expand deep wire inspector, and live controls (Protocol Switch, Chaos Sliders, Tamper Injector).
- **Admin Console (`/admin`):** Fleet inventory, Debian server metrics (CPU, RAM, DB), and security audit logs.
- **Driver Cockpit (`/driver, /alphamobile`):** Real-time HUD gauges, 4-wheel TPMS indicators, and AI voice guidance.

Subsystem Block	Key Internal Components	Input Interface / Output Interface	Hardware / Software Target
Block A: Firmware	FSM Mode Controller, Dynamics Engine, TPMS	Hardware ISR → JSON Object Array	ESP32 / FreeRTOS / C++
Block B: Crypto	Key Store, HMAC-SHA256, AES-GCM	JSON Object → Encrypted Wire Envelope	MbedTLS / BearSSL / Crypto
Block C: Gateway	HTTP Rest Adapter, Mosquitto MQTT	Wire Envelope → Network Sockets	WiFi 802.11 / Cellular 4G / TCP
Block D: Ingestion	Chaos Middleware, Crypto Validator, DB WAL	Network Stream → Persisted Records	FastAPI / Python 3.11 / SQLite
Block E: Semantic AI	Ollama Runtime, Prompt Template Engine	Parsed Sensor Vectors → Natural Text	Local GPU/CPU / Llama-3.2:3b
Block F: Presentation	WebSocket Multiplexer, NOC UI, Cockpit	SQLite / WS Events → Interactive DOM	Next.js 16 / React / Tailwind

CCNS CONCEPTS IN THE NOC PLATFORM AS BLOCKS

PAGE 5: CCNS BLOCKS

Course: 21ECC402P — Computer Communication and Network Security | NOC UI Mapping

Course Unit Alignment

Educational Purpose of the NOC Platform: The CVIS Network Operations Center (`/noc`) is not a passive monitoring dashboard; it is an active educational proving ground. Every slider, toggle, and inspection card corresponds directly to an academic concept from the CCNS curriculum (Units 1 to 5). The blocks below show how networking protocols, cryptographic mechanisms, access control models, and reliability theories are mapped to live, verifiable software controls.

BLOCK 1: Transport Protocol Agility

[CCNS UNIT 2: Transport Layer Architectures]

- **Dual Protocol:** HTTP/1.1 REST vs MQTT v3.1.1
- **Header Overhead:** ≈ 450 bytes (HTTP) vs 2 bytes (MQTT)
- **Live NOC Control:** Instant protocol switch without reboot
- **Wire Inspection:** Real-time protocol verification in 1.5s

BLOCK 2: Network Chaos Simulation

[CCNS UNIT 3: Flow Control & Reliability]

- **Packet Loss:** 0% → 25% Bernoulli drop simulation
- **Delay Injection:** 0 → 1000 ms queuing latency
- **Congestion Recovery:** Retransmission Timeout (RTO)
- **Live Control:** Real-time chaos sliders in NOC UI

BLOCK 3: Message Integrity (HMAC)

[CCNS UNIT 4: Message Integrity & Signatures]

- **HMAC-SHA256:** End-to-end cryptographic payload signing
- **Side-Channel Defense:** Constant-time digest comparison
- **Tampering Injector:** Flips bit in transit via NOC control
- **Enforcement:** Immediate HTTP 401 rejection & security log

BLOCK 4: Authenticated Encryption (AES)

[CCNS UNIT 4: Symmetric Ciphers & Confidentiality]

- **AES-128-GCM:** Confidentiality + data integrity
- **96-bit Unique IV:** Prevents keystream reuse
- **128-bit Tag:** Authenticates cleartext wire headers
- **Live Control:** Instant Plain JSON ↔ AES-GCM toggle

BLOCK 5: Access Control & Revocation

[CCNS UNIT 4: Authentication & Authorization]

- **Node Tokens:** Pre-shared cryptographic API keys
- **Dynamic Revocation:** Instant database status invalidation
- **Live Control:** One-click Disconnect / Reconnect in NOC
- **Lockout Result:** Immediate HTTP 401 gate & audit trail

BLOCK 6: Flow Observability & Tracking

[CCNS UNIT 1: Network Metrics & Flow Monitoring]

- **Waterfall Stream:** Sequence, protocol, size, status
- **Deep Inspector:** Headers, IV, auth tags, payload
- **Network Telemetry:** Live Round-Trip Time (RTT)
- **Live Control:** WebSocket streaming wire inspector

BLOCK 7: Anti-Replay Defense Logic

[CCNS UNIT 4: Network Attacks & Defense Models]

- **Timestamp Window:** Strict 30-second UTC freshness boundary
- **Monotonic Nonce:** 32-bit sequence counter per vehicle
- **Replay Mitigation:** Rejects stale or duplicate wire frames
- **Live Control:** Anti-Replay defense toggle switch in NOC

BLOCK 8: Service Decoupling Mode

[CCNS UNIT 3: High Availability & Degradation]

- **AI Kill-Switch:** Simulates cloud AI compute outage
- **High Availability:** Comms stays 100% operational
- **Architectural Proof:** Centralized AI rides on network
- **Live Control:** Stop / Start AI Service button in NOC

Demonstration Summary: The eight blocks above provide interactive, observable proof for theoretical networking and security concepts. In viva evaluation, each block can be toggled in real time on the live NOC console to show immediate packet rejections, latency spikes, or cryptographic transformations without requiring backend restarts.

EXPLANATION OF CCNS NOC BLOCKS

PAGE 6: VIVA & THEORY

Course: 21ECC402P — Computer Communication and Network Security | Theoretical Grounding

Faculty Viva Defense Reference

1. Transport Protocol Agility (CCNS Unit 2):
Theory: Explores tradeoffs between request-response application protocols (HTTP/1.1 over TCP) and lightweight publish-subscribe protocols (MQTT over TCP). HTTP introduces ≈ 450 bytes of ASCII headers per transmission, increasing radio duty cycle and cellular airtime cost. MQTT uses a 2-byte fixed header over persistent TCP sockets, reducing cellular overhead by 92%.
Implementation: Both adapters live behind an agnostic ingestion interface. The NOC toggle triggers `POST /api/v1/config/protocol`, broadcasting transport changes to the fleet. The node switches protocols at runtime without rebooting.
Viva Q&A: "Why not use MQTT exclusively?" → Corporate firewalls and cellular gateways frequently block non-standard ports (1883). Protocol agility guarantees seamless fallback to HTTP port 443 in restrictive enterprise environments.

2. Network Chaos & QoS Simulation (CCNS Unit 3):
Theory: Vehicular links suffer from channel fading, Doppler shift, and handover jitter. Replicating this typically requires expensive RF anechoic chambers or root-level Linux traffic control (`tc/netem`).
Implementation: Implemented as an ASGI middleware (`backend/middleware/chaos.py`) that intercepts raw HTTP requests prior to route handling. It executes a Bernoulli drop trial ($P_{\text{loss}} \in [0, 0.25]$) and injects non-blocking latency ($\tau \in [0, 1000$ ms$]$).
Viva Q&A: "How does the node survive packet drops?" → Firmware implements exponential backoff: $T_{\text{backoff}} = \min(T_{\text{base}} \times 2^{\text{retry}}, T_{\max})$, preventing channel congestion while guaranteeing delivery.

3. Cryptographic Integrity via HMAC (CCNS Unit 4):
Theory: Transport encryption (TLS) terminates at reverse proxies or cellular gateways, leaving payloads unprotected in backend transit. Message Authentication Codes (MACs) provide end-to-end cryptographic proof of authenticity and integrity.
Implementation: Payloads are signed with HMAC-SHA256(K_{dev}, M). Backend verifies using constant-time comparison (`hmac.compare_digest`) to prevent timing side-channels.
Live Demo: NOC "Packet Tampering" flips a single byte in transit. The backend rejects the frame with HTTP 401, logs `tamper_detected` in SQLite, and highlights the frame in red.

4. Authenticated Encryption via AES-GCM (CCNS Unit 4):
Theory: Traditional CBC mode ciphers provide confidentiality but lack built-in integrity, exposing them to padding-oracle attacks. Galois/Counter Mode (AES-128-GCM) delivers authenticated encryption with associated data (AEAD).
Implementation: Firmware encrypts payloads using a unique 96-bit Initialization Vector (IV). The envelope carries $\{\text{iv}, \text{ct}, \text{tag}\}$. The backend verifies the 128-bit authentication tag before deciphering the ciphertext.

5. Access Control & Instant Revocation (CCNS Unit 4):
Theory: Robust security requires dynamic credential revocation. When a vehicle node is compromised or hijacked, the central authority must terminate its access immediately without requiring fleet-wide rekeying.
Implementation: Clicking "Disconnect" sets `active = 0` in SQLite and publishes a control command. The backend rejects subsequent requests from that node with HTTP 401, demonstrating instant Zero-Trust authorization enforcement.

6. Deep Packet Inspection & Observability (CCNS Unit 1):
Theory: Real-time operational security demands wire-level inspection, Round-Trip Time (RTT) tracking, and packet decapsulation to detect anomalies, malformed frames, and denial-of-service patterns.
Implementation: The NOC packet table decapsulates raw wire frames in real time, revealing cryptographic flags, IVs, authentication statuses, payload sizes, and transmission latencies via an interactive click-to-expand inspector.

7. Anti-Replay Defense & Freshness (CCNS Unit 4):
Theory: Attackers can eavesdrop valid telemetry (e.g. "Vehicle Healthy") and replay it during a subsequent emergency (e.g. "Battery Overheating"), blinding the remote monitoring center.
Implementation: Every frame embeds a millisecond-precision UTC timestamp and a monotonic sequence counter. The backend enforces a strict 30-second freshness window and rejects duplicated or out-of-order sequence nonces.

8. Graceful Service Degradation (CCNS Unit 3):
Theory: In distributed systems, compute-tier failures must not cascade into transport or ingestion downtime (High Availability principle).
Implementation: Clicking "Stop AI Service" halts the Ollama neural inference pipeline. Telemetry ingestion, SQLite persistence, and WebSocket fan-out continue running with 100% reliability. This empirically proves the project's core thesis: **AI is an application layer riding on top of a resilient communication layer, not an embedded single point of failure.**

NOC Control Block	CCNS Syllabus Unit	Underlying Academic Concept	Live Demonstrable Action
Block 1: Protocol Agility	Unit 2: Protocol Stacks	Transport Layer Tradeoffs (HTTP vs MQTT)	Switch protocol; inspect header drop.
Block 2: Chaos Simulation	Unit 3: Flow & Congestion	Packet Loss, Latency Degradation, Retries	Increase loss to 10%; observe retry backoff.
Block 3: HMAC Integrity	Unit 4: Cryptography	Message Authentication Codes & Hashes	Inject tamper; observe immediate 401 reject.
Block 4: AES-GCM Envelope	Unit 4: Cryptography	Authenticated Symmetric Encryption	Toggle AES; observe ciphertext wire envelope.
Block 5: Revocation Model	Unit 4: Network Security	Device Authorization & Token Revocation	Click Disconnect; observe immediate lockout.
Block 6: Observability	Unit 1: Foundations	Network Metrics, Round-Trip Time, Wire Framing	Click packet; inspect raw headers and RTT.
Block 7: Anti-Replay	Unit 4: Network Security	Freshness, Sequence Numbers, Replay Defense	Resend old packet; backend rejects stale nonce.
Block 8: Decoupling	Unit 3: Resilience	Graceful Service Degradation & High Availability	Stop AI; verify telemetry continues streaming.